Exercici 2a

Variables

Una variable és un dipòsit on s'emmagatzema un valor. Consta d'un nom i pertany a un tipus (numèric, cadena de caràcters, etc.).

Una variable pot emmagatzemar:

- Valors Numèrics (100, 260, 1.24, 2.90, 5.00, etc.)
- Cadenes de caràcters ("Joan", "Compres", "Llistat", etc.).
- Valors lògics (*true*, *false*).
- Hi ha altres tipus de variables que veurem més endavant.

Les variables són noms que posem als llocs on emmagatzemem la informació. En *JavaScript*, han de començar per una lletra o un subratllat (_).

Una variable no pot tenir el mateix nom d'una paraula clau del llenguatge (Paraules reservades). En *JavaScript* tenim les següents paraules reservades:

Paraules reservades de *Javascript*				
abstract	*delete*	*function*	*Null*	*throw*
boolean	*do*	*goto*	*Package*	*throws*
break	*double*	*if*	*private*	*transient*
byte	*else*	*implements*	*protected*	*true*
case	*enum*	*import*	*public*	*try*
catch	*export*	*in*	*return*	*typeof*
char	*extends*	*instanceof*	*short*	*var*
class	*false*	*int*	*static*	*void*
const	*final*	*interface*	*super*	*while*
continue	*finally*	*long*	*switch*	*with*
debugger	*float*	*native*	*syncronized*	*true*
default	*for*	*new*	*this*	

Declaració de variables

Una variable es defineix anteposant la paraula clau var:

```
var dia;
```

Es poden declarar diverses variables en una mateixa línia:

```
var dia, mes, any;
```

A una variable se la pot definir i immediatament inicialitzar amb un valor:

```
var edat = 20;
```

o, si no en dos passos:

```
var edat;
edat = 20;
```

Tipus de variables

La forma en que se li assigna un valor a una variable depèn del tipus de valor que es vol emmagatzemar (nombres, textos, etc.)

1. **Variables numèriques**

   S'utilitzen per emmagatzemar valors numèrics enters (anomenats *integer* en anglès) o decimals (anomenats *float* en anglès). En aquest cas, el valor s'assigna indicant directament el nombre enter o decimal. Els nombres decimals utilitzen el caràcter. (Punt) en comptes de, (coma) per separar la part entera i la part decimal:

```
var iva = 16;           // variable tipus sencer
var total = 234.65;     // variable tipus decimal
```

2. **Cadenes de text**

   S'utilitzen per emmagatzemar caràcters, paraules i / o frases de text. Per assignar el valor a la variable, es tanca el valor entre cometes dobles o simples, per delimitar el seu començament i el seu final:

```
var missatge = "Benvingut al nostre lloc web";
var nomProducte = 'Producte ABC';
var lletraSeleccionada = 'c';
```

3. **Arrays**

   De vegades, als *arrays* se'ls anomena vectors, matrius i fins i tot arranjaments. No obstant això, el terme *array* és el més utilitzat i és una paraula comunament acceptada en l'entorn de la programació.

   Un *array* és una col·lecció de variables, que poden ser totes del mateix tipus o cadascuna d'un tipus diferent. La seva utilitat es comprèn millor amb un exemple senzill:

Si una aplicació necessita gestionar els dies de la setmana, es podrien crear set variables de tipus text:

```
var dia1 = "Dilluns";
var dia2 = "Dimarts";
...
var dia7 = "Diumenge";
```

Tot i que el codi anterior no és incorrecte, sí que és poc eficient i complica en excés la programació. Si en comptes dels dies de la setmana s'hagués de guardar el nom dels mesos de l'any, el nom de tots els països del món o els mesuraments diàries de temperatura dels últims 100 anys, s'haurien de crear desenes o centenars de variables.

En aquest tipus de casos, es poden agrupar totes les variables relacionades en una col·lecció de variables o *array*. L'exemple anterior es pot refer de la següent manera:

```
var dies = ["Dilluns", "Dimarts", "Dimecres", "Dijous", "Divendres", "Dissabte",
"Diumenge"];
```

Ara, una única variable anomenada dies emmagatzema tots els valors relacionats entre si, en aquest cas els dies de la setmana. Per definir un *array* s'utilitzen els claudators per delimitar el seu començament i el seu final i s'utilitza el caràcter, (coma) per separar els seus elements:

```
var nombre_array = [valor1, valor2, ..., valorN];
```

Un cop definit una matriu, és molt senzill accedir a cadascun dels seus elements. Cada element s'accedeix indicant la seva posició dins de l'*array*. L'única complicació, que és responsable de molts errors quan es comença a programar, és que les posicions dels elements comencen a comptar en el 0 i no en l'1:

```
var diaSeleccionat = dies[0];   // diaSeleccionat = "Dilluns"
var altreDia = dies[5];         // altreDia = "Dissabte"
```

En l'exemple anterior, la primera instrucció vol obtenir el primer element de l'*array*. Per a això, s'indica el nom de l'*array* i entre claudàtors la posició de l'element dins de l'*array*. Com s'ha comentat, les posicions es comencen a comptar en el 0, de manera que el primer element ocupa la posició 0 i s'accedeix mitjançant *dies[0]*.

El valor *dies[5]* fa referència a l'element que ocupa la sisena posició dins l'*array*. Com les posicions comencen a comptar a 0, la posició 5 fa referència al sisè element, en aquest cas, el valor '*dissabte*'.

4. Booleans

Les variables de tipus ***boolean*** o booleà també es coneixen amb el nom de variables de tipus lògic i el seu funcionament bàsic és molt senzill.

Una variable de tipus *boolean* emmagatzema un tipus especial de valor que només pot prendre dos valors: *true* (vertader) o *false* (fals). No es pot utilitzar per emmagatzemar nombres i tampoc permet guardar cadenes de text.

Els únics valors que poden emmagatzemar aquestes variables són ***true*** i ***false***. A continuació es mostra un parell de variables de tipus booleà:

```
var clientRegistrat = false;
var IVAinclos = true;
```

Durant el curs veurem la seva utilitat.

Elecció del nom d'una variable

Hem de triar noms de variables representatius. En l'exemple anterior els noms dia, mes, any són prou clars per donar-nos una idea sobre el seu contingut, una mala elecció de noms hagués estat cridar-les a, b i c. Podem donar-li altres bons noms. Altres no són tan representatius, per exemple d, m, a. Possiblement quan estiguem codificant un programa ens semblin noms representatius, però passat un temps l'oblidaríem.

Impressió de variables en una pàgina HTML

Per mostrar el contingut d'una variable en una pàgina podem utilitzar la propietat ***innerHTML*** de l'element on volem inserir el codi ***html***:

```
<!DOCTYPE html>
<html>

<head>
    <title>Exercici</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script>
        window.onload = function () {
            var nom = 'Joan';
            var edat = 10;
            var alçada = 1.92;
            var casat = false;

            var res = '<h1>Impressió de variables en una pàgina HTML</h1>';
            res += '<br>';
            res += nom;
            res += '<br>';
            res += edat;
            res += '<br>';
            res += alçada;
            res += '<br>';
            res += casat;

            document.getElementById("resultat").innerHTML = res;
        };
    </script>
</head>

<body>
    <div id="resultat"></div>
</body>

</html>
```

Quan imprimim una variable, no l'hem de disposar entre cometes (en cas de fer això, apareixerà el nom de la variable i no el seu contingut).

Els valors de les variables en les que emmagatzemem noms (és a dir, són cadenes de caràcters) han d'anar tancades entre cometes. Els valors de les variables numèriques (en aquest exemple les variables *edat* i *alçada*) no han d'anar tancades entre cometes. Cada instrucció finalitza amb un punt i coma.

Les variables de tipus *boolean* poden emmagatzemar només dos valors: *true* o *false*.

El resultat en visualitzar la pàgina ha de similar a aquest:

> ***Impressió de variables en una pàgina HTML***
> *Joan*
> *10*
> *1.92*
> *false*

És a dir que es mostren els continguts de les 4 variables. Una variable és d'un tipus determinat quan li assignem un valor.

Operador *typeof*

typeof és un operador que et diu el tipus de dada d'una expressió o variable.

Exemples:

```
typeof 42;          // "number"
typeof "Hola";      // "string"
typeof true;        // "boolean"
```

Maneres modernes de declarar variables

Què són *let* i *const*?

Són maneres modernes de declarar variables (*ES2015*+), pensades per substituir molts usos de *var*.

- ***let***: variable **reassignable** amb **àmbit de bloc**.
- ***const***: variable **no reassignable** amb **àmbit de bloc**.

Àmbit de bloc vol dir que només existeixen dins { ... } (funcions, *if*, *for*, etc.).

Diferències entre *let/const* i *var*

1) Àmbit (*scope*) de bloc

```
        if (true) {
            let x = 1;
            const y = 2;
        }
        console.log(typeof x, typeof y); // "undefined undefined" → no existeixen
fora del bloc
```

Amb *var*, en canvi, tindries àmbit de **funció** (o global) i és més fàcil "embrutar" l'espai global.

2) Reassignació vs mutació

```
        let a = 1;
        a = 2;            // OK

        const b = 1;
        b = 2;            // ❌ TypeError: no pots reassignar un const
```

Però *const no vol dir immutable*. Pots **mutar** el contingut d'un objecte/array declarat amb const:

```
        const user = { nom: "Ada" };
        user.nom = "Lovelace"; // ✅ mutació OK
        // user = {}           // ❌ reassignació NO

        const nums = [1, 2];
        nums.push(3);         // ✅ mutació OK
```

Per impedir mutacions, es pot utilitzar *Object.freeze()* o estratègies d'immutabilitat.

3) "*Hoisting*" i Temporal Dead Zone (*TDZ*)

Les declaracions amb ***let*** i ***cons***t es *pugen* (*hoist*) internament, però **no** es poden usar abans de la línia on les inicialitzes (*TDZ*).

```
console.log(x); // ❌ ReferenceError (TDZ)
let x = 10;

console.log(y); // ❌ ReferenceError (TDZ)
const y = 20;
```

Amb *var*, obtindries *undefined* en comptes d'un error, cosa que pot amagar *bugs*.

Quan utilitzar *let* i quan *const*?

Regla pràctica:

- ***const* per defecte.** Declara amb *const* sempre que **no** necessitis reassignar la variable. Dona més seguretat i comunica intenció.
- ***let*** només quan **cal reassignar** (com comptadors, acumuladors, canvis d'estat temporals).

Exemples breus d'ús

1) Estat que canvia → *let*

```
let total = 0;
for (let i = 0; i < prices.length; i++) {
    total += prices[i];
}
```

2) Constants i funcions → *const*

```
const PI = Math.PI;
const round2 = function (n) {
    return Math.round(n * 100) / 100
};
const areaCercle = function (r) {
    return round2(PI * r * r)
};
console.log(areaCercle(10.5468));   // 349.46
```

3) Objectes i *arrays* amb *const* (mutació permesa)

```
const config = { retries: 3, timeoutMs: 5000 };
config.retries = 5;          // OK
// config = { ... }          // NO

const items = [];
items.push("nou");           // OK
```

Exercicis

Declaració de variables amb *var*

1. Confeccionar un programa en *JavaScript* on haureu de definir una variable del tipus cadena de caràcters per desar el nom d'un treballador i una altra variable del tipus numèrica per desar el seu sou. Imprimir cada variable en una línia diferent a la pantalla.

2. Defineix un *array* amb els mesos de l'any i imprimeix només els mesos imparells.

Declaració de variables amb *let* i *const*

3. Crea una pàgina amb un botó. Cada vegada que es faci clic al botó s'actualitzarà un comptador de *clicks* i es mostrarà en un paràgraf.

4. Confecciona un programa que calculi el perímetre d'una circumferència a partir d'un radi i que mostri el resultat en un paràgraf.